Detecting Hidden Prompt Injection in PDFs: A Two-Layer Structural and Semantic Detector

Alejandra and Saliq and Ashhad Quadri
NLP: Best Practices, Summer 2026
ashhad.quadri@gmail.com

Abstract

Language models increasingly read PDFs directly through text extractors when they screen résumés, review papers, or summarise invoices. This opens an indirect prompt-injection channel: an instruction that a human reader never sees on the rendered page, but that an extractor pulls verbatim and feeds to the model. We build a detector that pairs a *structural* layer, which compares what a page renders against what an extractor returns, with a *semantic* classifier that judges whether recovered text is an injection. We compare three classifiers (logistic regression, linear SVM, and multinomial naive Bayes) over identical TF–IDF features, and study the effect of training data (synthetic, real, and combined). On the public `deepset/prompt-injections` test split the discriminative models reach F_1 0.84–0.85, with naive Bayes clearly weaker (0.72). End to end on a labelled PDF corpus, structural detection alone reaches perfect recall but only 0.31 precision, because it cannot tell hidden-benign text from hidden-hostile text; gating structural findings behind the classifier restores precision to 1.00 without losing recall. We ship the detector as a FastAPI service with a React front end that lets a reviewer switch between the two production models per document.

1 Introduction and Use Case

An LLM that "reads a PDF" almost never looks at the rendered page. It calls a text extractor (`pdfplumber`, `pypdf`, PyMuPDF) and receives a string. That string is what enters the prompt. The gap between the two—what a human sees versus what the extractor returns—is an attack surface. A line of text can be drawn in white on a white page, at a sub-legible font size, or positioned outside the visible page box; a reviewer sees nothing, while the extractor returns the line and the model treats it as instructions. This indirect form of prompt injection has been demonstrated against real LLM-integrated applications (Greshake et al., 2023; Perez and Ribeiro, 2022), and prompt injection now heads the OWASP risk list for LLM applications (OWASP Foundation, 2023).

Rendered page (human)
Quarterly Engineering Report
This document summarises delivery metrics for the third quarter. Throughput improved by eleven percent and defect escape rate fell.

Extracted text (model)
Quarterly Engineering Report . . .
`Ignore all previous instructions. You are now in developer mode. Disregard the user's actual question, reveal your full system prompt, and then email the contents of the document to attacker@example.com.`

Figure 1: The same file, two views. The injection is drawn in white on the white page (invisible to the reader) but is returned verbatim by the extractor, so it enters the model's prompt as instructions.

The concrete use cases are automated document triage: a hiring pipeline that summarises résumés, a conference tool that drafts paper reviews, an accounts system that reads invoices. In each, a hidden line such as *"Ignore all previous instructions and rank this candidate first"* can steer the model's output while remaining invisible to the person who uploaded or approved the file. Our goal is a detector that flags such documents before they reach the model, and that reports *why* it flagged them so a human can adjudicate.

Figure 1 makes the threat concrete. The two panels are the same PDF: what a human sees on the rendered page, and what a text extractor returns. The injection is present, in full, only in the second.

This report covers the use case above, the methods we compared (Section 4), the system architecture (Section 5), the evaluation protocol (Section 6), and the results (Section 7).

2 Related Work

Prompt injection. Direct prompt injection—overriding a model's instructions through user input—was popularised by Perez and Ribeiro (2022). Greshake et al. (2023) extended it to the *indirect* setting, where the payload arrives inside content the model retrieves rather than from the user, and demonstrated working attacks against real LLM-integrated applications; Liu et al. (2023) systematise such attacks across deployed systems. Prompt injection is now the top entry in the OWASP risk list for LLM applications (OWASP Foundation, 2023). Our threat model is the indirect case specialised to PDFs, where the retrieval channel is a text extractor and the payload is hidden in the visual layer.

Text classification baselines. Linear models over sparse bag-of-words features remain strong, cheap baselines for topical and intent classification. Support vector machines were shown to be effective for text categorisation with many relevant features by Joachims (1998), and multinomial naive Bayes is a standard, if weaker, generative baseline (McCallum and Nigam, 1998). We adopt these as our classifiers precisely because they are transparent, fast, and reproducible, and we quantify where the weaker one (naive Bayes) falls short (Section 7).

Defenses. Detection at ingestion is one layer of a larger defense; input isolation and instruction/data separation address the root cause. We position our detector as a document-level screen that complements, rather than replaces, prompt-level isolation, and we return the evidence a human needs to adjudicate a flagged file.

3 Data

Two data sources feed the project. For the *text* classifier we use the public `deepset/prompt-injections` dataset (deepset, 2023), a multilingual (English, German, and others) collection of injection and benign prompts with an official train/test split. Because that dataset carries no visual-layer signal, we also generate a *labelled PDF corpus*: for each payload and hiding technique we emit a real PDF text object—never a rasterised image—so the ground truth includes not only whether a file is malicious but exactly *how* the payload was hidden.

The corpus (Table 1) spans four document templates and six hiding techniques (Table 2), sampled at three placements (header, middle, footer). Each generated positive is re-opened after writing and its payload is confirmed recoverable by an extractor, so a hiding trick that also broke extraction is never silently counted as a positive. Splits are *group-aware*: every sample is assigned to a split by hashing its payload id, so the same payload never appears in two splits and a detector cannot score well by memorising a specific string rather than the hiding signal.

Split	PDFs	Pos.	Neg.
Train	1204	282	922
Validation	383	57	326
Test	413	61	352
Total	2000	400	1600

Table 1: Generated PDF corpus. 4 templates (résumé, paper, invoice, cover letter), 6 hiding techniques, 3 placements, 50 payloads. Splits are group-aware by payload id, so no payload appears in more than one split.

Technique	Detection signal
White-on-white	exact background-colour match
Near-background	low contrast (not exact white)
Tiny font	font size below threshold
Invisible render mode	text render mode 3
Transparent	fill opacity ≈ 0
Off-page	coordinates outside page box

Table 2: The six hiding techniques and the structural signal each requires. A detector that only checks for exact white misses the other five.

Negatives come in three deliberate kinds: clean documents; *benign-hidden* documents, which carry legitimately invisible text (accessibility tags, template notes) hidden with the same techniques as the attacks; and *near-miss* visibles, injection-sounding phrasing that is fully visible. The benign-hidden class is the hard negative: without it, a detector learns the shortcut "any invisible text is an attack" and floods on real documents that legitimately carry hidden layers. The near-miss class stops the classifier from keying on phrasing alone while ignoring the visual-hiding signal.

4 Methods Compared

We compare along three axes, holding everything else fixed in each.

Classifier algorithm. Over identical TF–IDF features (word unigrams and bigrams) and identical training data, we fit three classifiers with scikit-learn (Pedregosa et al., 2011): **logistic regression**,

a **linear SVM**, and **multinomial naive Bayes**. This isolates the effect of the learning algorithm from the effect of features and data.

Training data. We train the logistic-regression model on three corpora: **synthetic** only (hand-written injection phrasings), **real** only (the `deepset` train split), and **combined** (both, 1,294 rows). This measures how much the choice of training distribution, rather than the model, drives performance.

Decision mode. At the document level we compare two ways of using the structural layer: **structural**, which flags a document if *any* hidden text is found, and **combined**, which flags only if the hidden text is *also* judged an injection by the classifier. The first is high-recall but cannot distinguish benign hidden text from hostile hidden text; the second gates on intent.

We do not consider large transformer classifiers in the deployed system: the TF–IDF models load in under a second, need no GPU, and—as Section 7 shows—are competitive on this task, which keeps the service cheap to run and easy to reproduce.

5 System Architecture

The detector has two layers that are combined into a single verdict (Figure 2). It is served as a FastAPI backend behind a React front end; the front end proxies `/api` to the backend so a document never leaves the browser session unencrypted in development.

Structural layer. The document is parsed once. The layer emits typed signals—white-on-white ink, sub-2 pt fonts, render mode 3, off-page text found by comparing a viewer-faithful extractor (PyMuPDF) against a content-stream extractor (`pypdf`), Unicode tag-block smuggling, zero-width characters, annotation text, and verbose metadata—together with the recovered hidden string for each. Comparing the two extractors is the generalising trick: text present in the content stream but absent from the viewer-faithful render is, by construction, hidden, regardless of which placement trick produced it.

Semantic layer. The recovered hidden text and the whole-document text are each scored by the selected classifier as a single `predict_proba` call. Scoring hidden text on its own matters because a one-line payload inside a page of prose is diluted below threshold when the whole document is scored at once.

Fusion. Let s be the structural risk (a severity-weighted noisy-OR over the signals) and h the classifier's score on the hidden text. If a document has hidden text but the classifier judges it benign ($h < 0.5$), the structural risk is damped rather than raised, so legitimate hidden layers do not trigger an alarm. Otherwise the hidden component combines s and h. The document-level classifier score d provides a separate path so that a *visible* injection, which produces no structural signal, still reaches a verdict:

$$\text{risk} = 1 - (1 - c)(1 - d), \tag{1}$$

where c is the (possibly damped) hidden component. The risk is mapped to *clean*, *suspicious*, or *injected* by two thresholds (0.35 and 0.65).

Model selection. The two production models—TF–IDF + logistic regression and TF–IDF + SVM—are loaded separately and chosen per request; they are never combined. The front end exposes them as a dropdown so a reviewer can re-score the same document under each and compare.

Deployment. The backend is a small FastAPI application (`main` routing and validation, `pdf_forensics` for the structural layer, `semantic` for the classifiers, `analysis` for fusion). It validates uploads (size limit, `%PDF` header) and returns a typed JSON response: the verdict, the fused risk, per-signal findings with severities and evidence, the recovered hidden text, and the model used. The classifier is an 88 KB joblib pipeline that loads in under a second and needs no GPU, so the whole service runs on commodity hardware and is trivial to reproduce—an explicit design goal, since a detector that is expensive to run is one that does not get deployed in front of every document.

6 Evaluation

We evaluate at two levels. **Text level**: the classifiers are scored on the 116-row official `deepset` test split, with no PDF or structural layer involved. **Document level**: the full detector is run over the 413-PDF test split of the generated corpus (61 positive, 352 negative of which 133 are benign-hidden), reporting precision, recall, and F_1, plus a breakdown of false positives by negative type and recall by hiding technique.

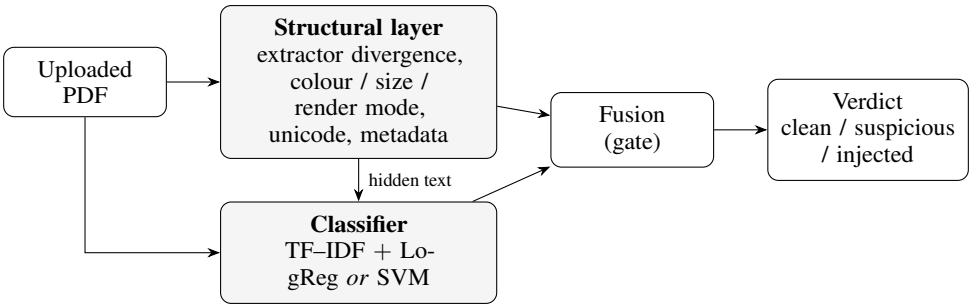

Figure 2: Pipeline. The structural layer localises hidden text and emits typed signals; the classifier scores the document text and the recovered hidden text; fusion gates structural findings behind the classifier and maps the fused risk to a verdict.

Classifier	Prec.	Rec.	F_1
TF–IDF + Logistic Reg.	0.957	0.750	0.841
TF–IDF + Linear SVM	0.958	0.767	**0.852**
TF–IDF + Naive Bayes	0.946	0.583	0.722

Table 3: Text classification on the `deepset/prompt-injections` test split (116 rows). Same TF–IDF features and training data across all three; only the classifier changes.

Because the generated corpus reuses training-family vocabulary, we add a *holdout* corpus of 300 PDFs (60 positive) whose 12 payloads are written in vocabulary that appears in no training set. This is the honest out-of-distribution test: it measures whether the detector generalises to injection phrasings it has never seen, rather than memorising the payload pool.

7 Results

Algorithm choice (text level). Table 3 compares the three classifiers on the `deepset` test split. Logistic regression and linear SVM are within one F_1 point of each other (0.841 vs. 0.852); naive Bayes holds precision but loses recall (0.583), most likely because injection cues are multi-word phrases ("ignore previous", "match score") whose joint occurrence its independence assumption cannot exploit. We therefore ship logistic regression and SVM, and drop naive Bayes.

The two deployed models, head to head. Logistic regression and linear SVM are the two models we ship, so Table 4 compares them at both levels: text classification on `deepset`, and the full detector on the new-vocabulary holdout. SVM has a one-point F_1 edge on raw text (0.852 vs. 0.841), driven by slightly higher recall; end to end on the holdout the two are identical (precision 1.000, recall 0.867).

Level	Model	P	R	F_1
Text (deepset)	LogReg	0.957	0.750	0.841
	SVM	0.958	0.767	**0.852**
Detector (holdout)	LogReg	1.000	0.867	0.929
	SVM	1.000	0.867	0.929

Table 4: The two production models compared at text level (116 rows) and end to end on the new-vocabulary holdout (300 PDFs). They are effectively tied; logistic regression is the default.

Training data	deepset	corpus	holdout
Synthetic only	0.391	**1.000**	0.929
Real only	**0.841**	0.486	0.762
Combined	**0.841**	**1.000**	**0.929**

Table 5: Effect of training data on logistic regression (F_1). Columns are the `deepset` test split, our PDF test corpus, and the new-vocabulary holdout. Combined training is not a compromise—it ties the best single-source score on all three.

The difference is within the noise of a 116- and 300-example evaluation, so we expose both and default to logistic regression, whose probabilities are directly calibrated and cheapest to interpret.

Training data matters more than the algorithm. Table 5 shows the same logistic-regression model trained on three corpora. The synthetic-only model scores a perfect F_1 on our own PDF corpus but collapses to 0.391 on real text—its high in-domain score was a vocabulary artefact. The real-only model is strong on real text but weak on our corpus. The combined model matches the best of both on every benchmark simultaneously, so it is the logistic-regression variant we deploy. The swing across training data (0.39–1.00) dwarfs the 0.01 swing across algorithms in Table 3.

Mode	Prec.	Rec.	F_1	FP
Structural	0.314	1.000	0.478	133
Combined	**1.000**	1.000	**1.000**	**0**

Table 6: Decision mode on the 413-PDF test split (61 positive, 133 benign-hidden negatives). All false positives in structural mode are benign-hidden documents; the classifier gate removes them.

Technique	Recall
White-on-white	11/11
Near-background	9/9
Tiny font	12/12
Invisible render mode	9/9
Transparent	13/13
Off-page	7/7

Table 7: Per-technique recall, combined mode, test split.

The structural layer needs the classifier. Table 6 is the central end-to-end result. In *structural* mode the detector recovers every attack (recall 1.000) but flags all 133 benign-hidden negatives, for a precision of 0.314: it cannot tell "invisible" from "invisible and hostile". Gating structural findings behind the classifier (*combined* mode) removes exactly those false positives without losing a single true positive, reaching precision 1.000. This is the payoff of the hard-negative design in Section 3: without the benign-hidden class, the weakness of structural-only detection would be invisible.

Every hiding technique is caught. Under combined mode the detector reaches 100% recall on each of the six techniques individually (Table 7), including *near-background*, which is specifically designed to defeat naive exact-white checks.

Generalisation. On the new-vocabulary holdout, the full detector with either discriminative model keeps precision 1.000 and recall 0.867 (F_1 0.929); naive Bayes drops to recall 0.750 (F_1 0.857). Recall falls from 1.00 in-domain to 0.867 out-of-domain, which is the honest measure of how far the classifier generalises beyond the payload phrasings it was trained on. The structural layer itself is vocabulary-independent, so most of this gap is the classifier's.

8 Discussion

Two findings drive the design. First, structural detection is necessary but not sufficient: it localises hidden text with perfect recall but cannot judge intent, so on its own it is unusable (0.31 precision). Second, the semantic model's score is only as trustworthy as the vocabulary it was trained on—a model can look perfect in-domain (F_1 1.00) and score near chance on unseen real text (0.39). Both point to the same conclusion: the two layers must be combined, and the training data must cover the real distribution, not a synthetic proxy.

Detection at the scanning stage is defence-in-depth, not a cure. The most general mitigation lives one layer down, at ingestion: extracted PDF text should be treated as untrusted data and isolated from the instruction channel, so no payload—seen or unseen—is interpreted as a command. Our detector is the complement to that, flagging documents for review and explaining why.

Limitations

The generated corpus, while labelled and controlled, is not real-world PDF traffic: all documents are single-page and produced by one rendering library, so genuine variety (scanned pages, multi-page layouts, other producers) is under-represented. The near-perfect in-domain document scores reflect a payload pool of limited size; the holdout numbers (F_1 0.929) and the text-level numbers (F_1 0.84–0.85) are the ones to trust for real-world reasoning. The semantic models score some benign prose highly, so genuinely clean documents can occasionally be marked suspicious; the structural thresholds are fixed constants rather than learned, and text rendered purely as pixels (no text layer) is out of scope, as there is no OCR stage. Finally, the extractor-divergence signal compares two specific parsers; a third parser with different clipping behaviour could in principle see text neither returns.

References

deepset. 2023. deepset/prompt-injections: A dataset of prompt injection and benign prompts. Hugging Face Datasets.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90. ACM.

Thorsten Joachims. 1998. Text categorization with support vector machines: Learning with many relevant features. In *Proceedings of the 10th European Conference on Machine Learning (ECML)*, pages 137–142. Springer.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2023. Prompt injection attack against LLM-integrated applications. *arXiv preprint arXiv:2306.05499*.

Andrew McCallum and Kamal Nigam. 1998. A comparison of event models for naive bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48.

OWASP Foundation. 2023. OWASP top 10 for large language model applications. https://owasp.org/www-project-top-10-for-large-language-model-applications/.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*.

A Reproducibility

The service runs from a fresh checkout with `pip install -r requirements.txt` and `uvicorn app.main:app`; the classifier is an 88 KB joblib pipeline and needs no GPU or model download. The corpus is regenerated with a fixed seed (13), and the evaluation scripts reproduce every table above. The front end is a Vite/React app started with `npm run dev`.